Estimation
&
Design Principles

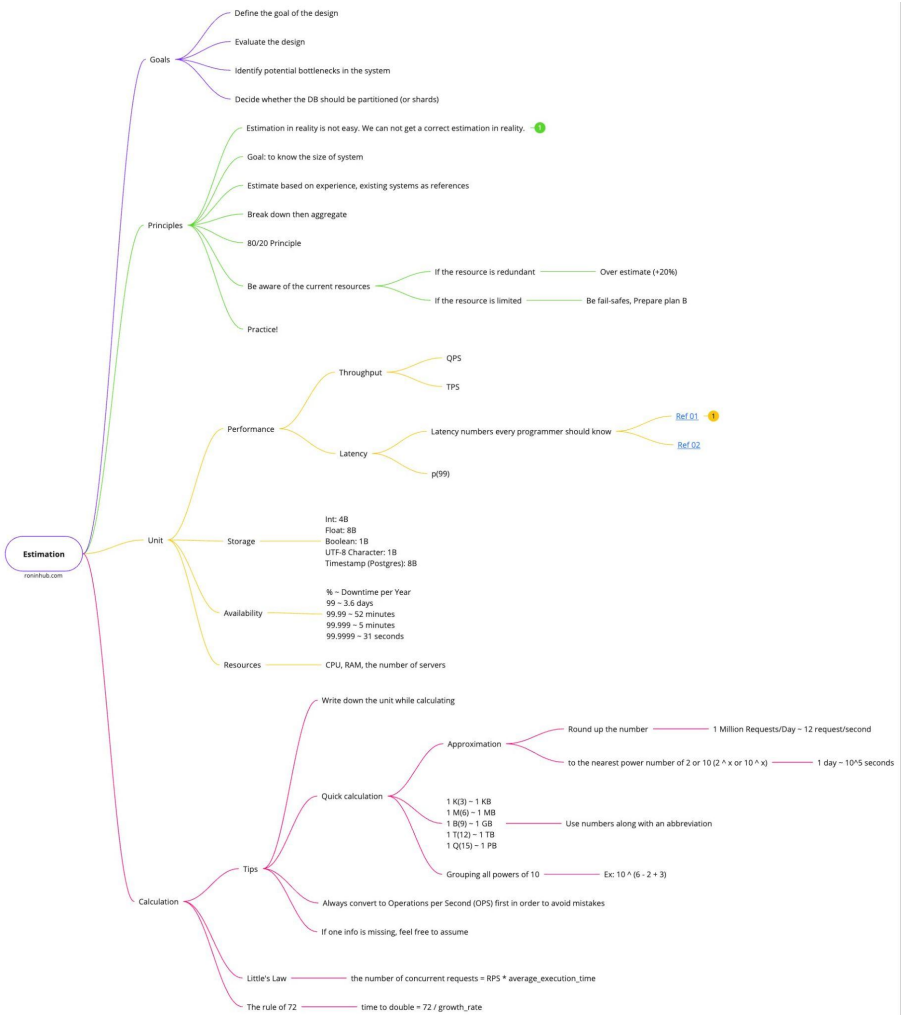
“Tìm những người mày mang ơn trả.
Mọi thứ cứ để nhân quả”
– Mở Mắt (Lil Wuyn, Đen)



Outline

1. Estimation
2. Trade-off
3. Design Principles
    - Fundamentals
    - Goals
        i. Scalability
        ii. Performance
        iii. Availability

1. Estimation


Estimation
roninhub.com
Goals
Define the goal of the design
Evaluate the design
Identify potential bottlenecks in the system
Decide whether the DB should be partitioned (or shards)
Principles
Estimation in reality is not easy. We can not get a correct estimation in reality.
Goal: to know the size of system
Estimate based on experience, existing systems as references
Break down then aggregate
80/20 Principle
Be aware of the current resources
If the resource is redundant
Over estimate (+20%)
If the resource is limited
Be fail-safes, Prepare plan B
Practice!
Unit
Performance
Throughput
QPS
TPS
Latency
Latency numbers every programmer should know
Ref 01
Ref 02
p(99)
Storage
Int: 4B
Float: 8B
Boolean: 1B
UTF-8 Character: 1B
Timestamp (Postgres): 8B
Availability
% ~ Downtime per Year
99 ~ 3.6 days
99.99 ~ 52 minutes
99.999 ~ 5 minutes
99.9999 ~ 31 seconds
Resources
CPU, RAM, the number of servers
Calculation
Tips
Write down the unit while calculating
Quick calculation
Approximation
Round up the number
1 Million Requests/Day ~ 12 request/second
to the nearest power number of 2 or 10 (2 ^ x or 10 ^ x)
1 day ~ 10^5 seconds
1 K(3) ~ 1 KB
1 M(6) ~ 1 MB
1 B(9) ~ 1 GB
1 T(12) ~ 1 TB
1 Q(15) ~ 1 PB
Use numbers along with an abbreviation
Grouping all powers of 10
Ex: 10 ^ (6 - 2 + 3)
Always convert to Operations per Second (OPS) first in order to avoid mistakes
If one info is missing, feel free to assume
Little's Law
the number of concurrent requests = RPS * average_execution_time
The rule of 72
time to double = 72 / growth_rate

Exercise 1: URL Shortener

Questions:

1. What is the average number of writes per day?
2. What is the read:write ratio?
3. What is the length of a short URL?
4. How many years should the short URL be persisted?

Answers:

1. 10M writes per day
2. 100:1
3. At most 8 characters
4. 4 years

Exercise 1: URL Shortener / Traffic

- 10M writers per day → 10^7 TPD / 10^5 seconds = 10^(7-5) = 100 TPS
- Read:write = 100:1 → QPS = TPS * 100 = 10K QPS

Exercise 1: URL Shortener / Storage

- A record
  - Long URL: 1 KB
  - Short URL: 10 B
  - User_id: 16 B
  - Created_at: 8 B
  - Expires_at: 8 B
  - Updated_at: 8 B
  - → a record ~ 1.1 KB
- Storage for 4 years: 1.1 KB/write * 10M writes/day * 400 days * 4 years = (1.1 * 4 * 4) * 10 ^ (3 + 7 + 2) = 17.6 * 10^12 = 17.6 TB
- Replication Factor: 3 → total storage = 3 * 17.6 TB = 52.8 TB

Exercise 1: URL Shortener / Memory

- TTL: 1 days
- Apply 80/20 principles
- 10M writes/day * 1 day * 1.1 KB/record * 0.2

  = (0.2 * 1.1) * 10^(7+3) = (2 * 1.1) * 10^9 = 2.2 GB
- Tip: M * K = G

Exercise 1: URL Shortener / Bandwidth

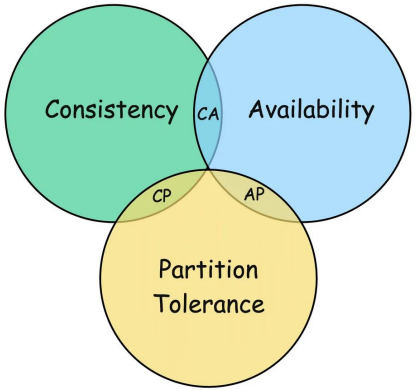
- Read:write = 100:1 → ignore bandwidth for write operation
- 10K QPS
- Assume the average size of a GET request that not includes token/cookies: 1 KB
- Ingress: 10K QPS * 1 KB → 10 MB/s
- Egress: 10K QPS * (1+1.1) KB → 21 MB/s

2. Trade-Off

“Everything in software architecture is trade-off” - Neal Ford

CAP

- Any distributed system can simultaneously provide at most two of three guarantees: Consistency, Availability, and Partition tolerance (CAP)
- What is the difference between C in CAP and C and ACID?



Common Trade-offs

- Consistency vs Availability (CAP)
- Consistency vs Performance
  - Synchronous vs Asynchronous
  - Strong vs Eventual Consistency
- Space vs Time:
  - Memory vs CPU / IO
  - Caching
  - Compression
  - Normalization vs Aggregation
  - Resources vs Latency
    - Batch vs Stream processing
- Performance vs Security
- Complexity vs Flexibility
- Development Speed vs Quality

Resolving Trade-off

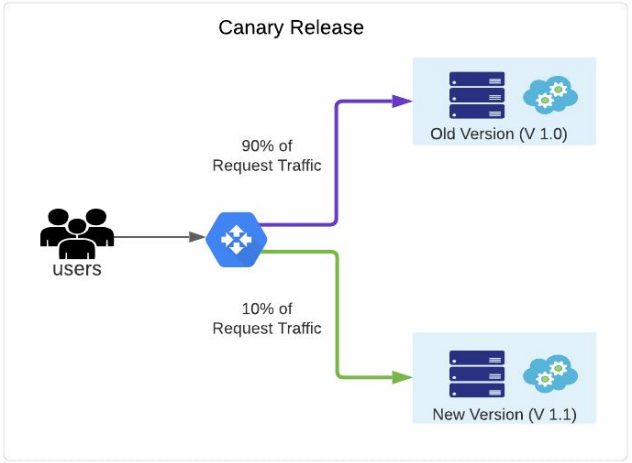
- Some trade-offs can be mitigated only or overcomed.
- The root cause of many trade-offs are **couplings.**
- **Separation**: Resolve the coupling, you can resolve the trade-off.
- How to find couplings?
    - Dependency / Relationship / Reuse / Constant
    - TRIZ: Separation Techniques



Separation in Time

- Modern Umbrella
- Startup (Agility, Cost) → Enterprise (Reliability, Security)
- Auto scale out
- Canary deployment: A/B testing, monitoring perf

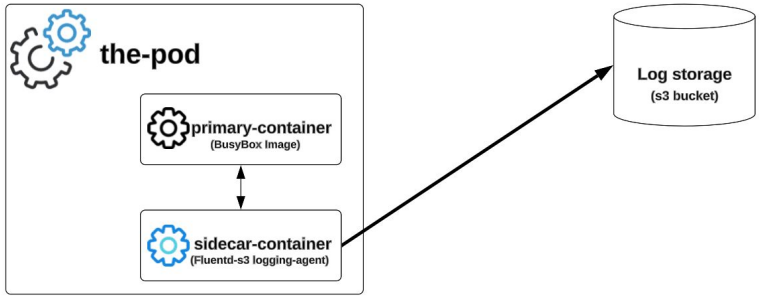




Separation in Space

- Knife, Hammer
- Sidecar pattern
- Each microservice is optimized differently
  - Product Catalog service uses Elasticsearch
  - Order service uses Postgresql
- Multi-region deployments
- Each schema are used for each client (tenant)

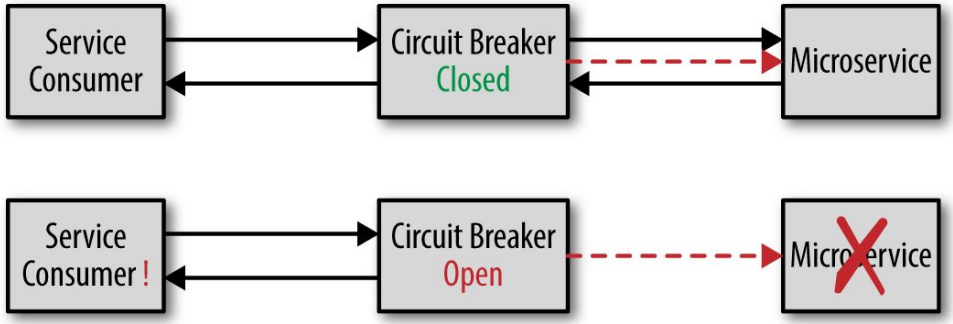




Separation between Parts and Whole

- Concept: A system's properties can be split between the whole object and its individual parts. **The requirements for the entire system may differ from those for each component.**
- Bicycle chain
- Each microservice have low fault tolerance but the whole microservices has high fault tolerance, high elasticity.





Concern Points

- List of elements
  - Duplication
  - Order
- Relationship: Direction
- Consistency

3. Design Principles

Fundamental Principles

Fundamental Principles

- Define goals first
- Be aware of limitations
  - Every technology has limitation, even the pros have limitation
  - Evaluate the limitation and our needs
- Trade-off
- Avoid premature optimization
- SOLID
- 80/20 principle
- KISS (don’t make me think) - try to avoid over-engineering
- **Understand principle of operation (of existing solutions) to design**

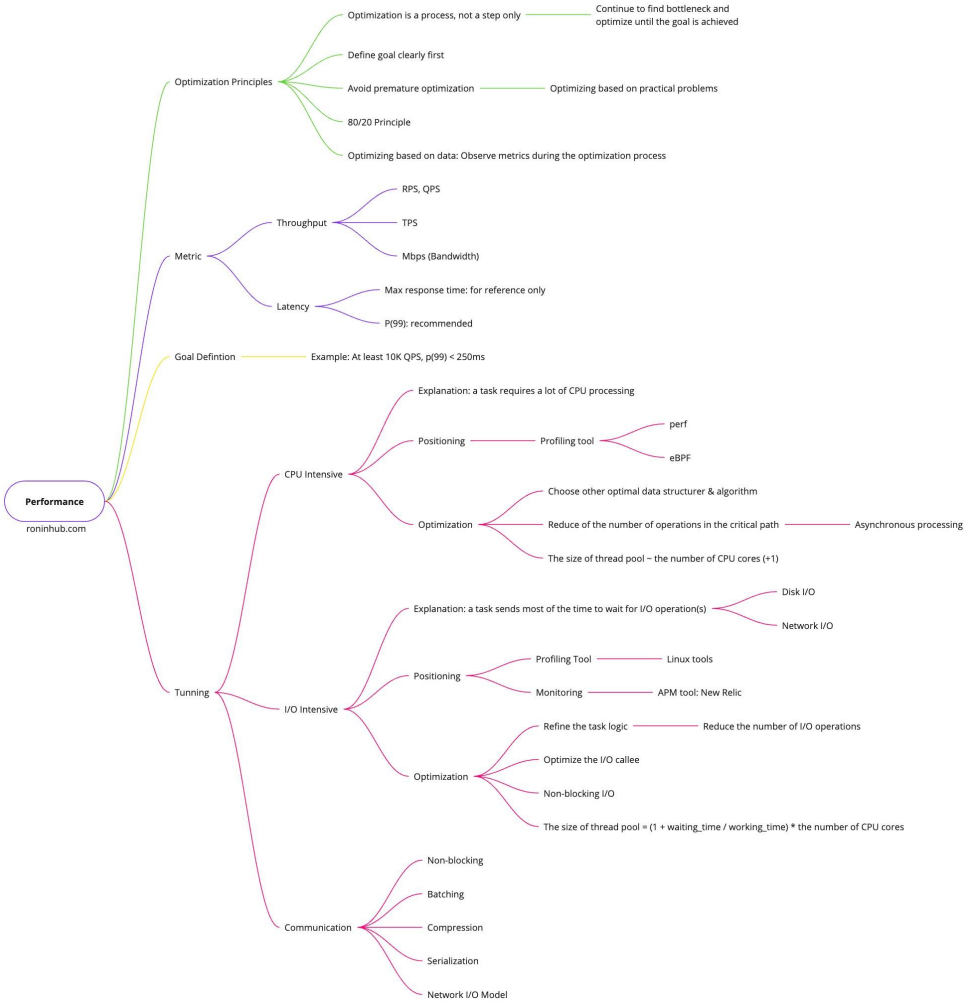
Goal: Scalability

What factors affect to scalability?

Scalability



Goal: Performance


Performance
roninhub.com
Optimization Principles
Optimization is a process, not a step only
Continue to find bottleneck and optimize until the goal is achieved
Define goal clearly first
Avoid premature optimization
Optimizing based on practical problems
80/20 Principle
Optimizing based on data: Observe metrics during the optimization process
Metric
Throughput
RPS, QPS
TPS
Mbps (Bandwidth)
Latency
Max response time: for reference only
P(99): recommended
Goal Defintion
Example: At least 10K QPS, p(99) < 250ms
Tunning
CPU Intensive
Explanation: a task requires a lot of CPU processing
Positioning
Profiling tool
perf
eBPF
Optimization
Choose other optimal data structurer & algorithm
Reduce of the number of operations in the critical path
Asynchronous processing
The size of thread pool ~ the number of CPU cores (+1)
I/O Intensive
Explanation: a task sends most of the time to wait for I/O operation(s)
Disk I/O
Network I/O
Positioning
Profiling Tool
Linux tools
Monitoring
APM tool: New Relic
Optimization
Refine the task logic
Reduce the number of I/O operations
Optimize the I/O callee
Non-blocking I/O
The size of thread pool = (1 + waiting_time / working_time) * the number of CPU cores
Communication
Non-blocking
Batching
Compression
Serialization
Network I/O Model

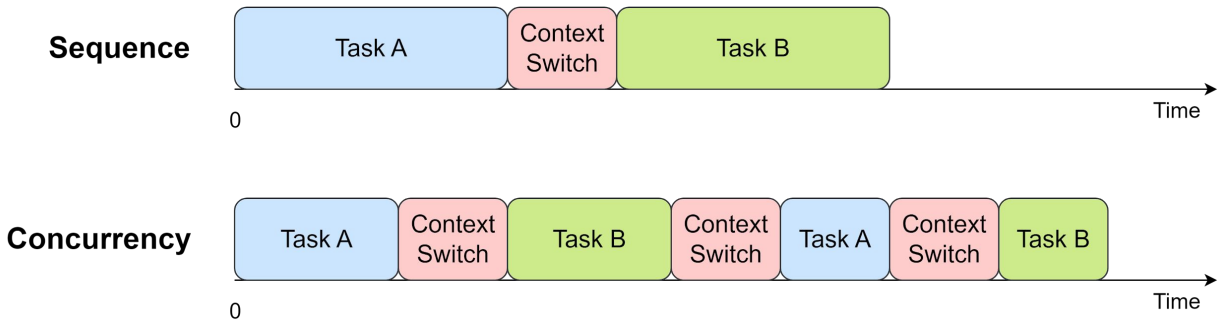
Finding Bottleneck

Finding Bottlenecks

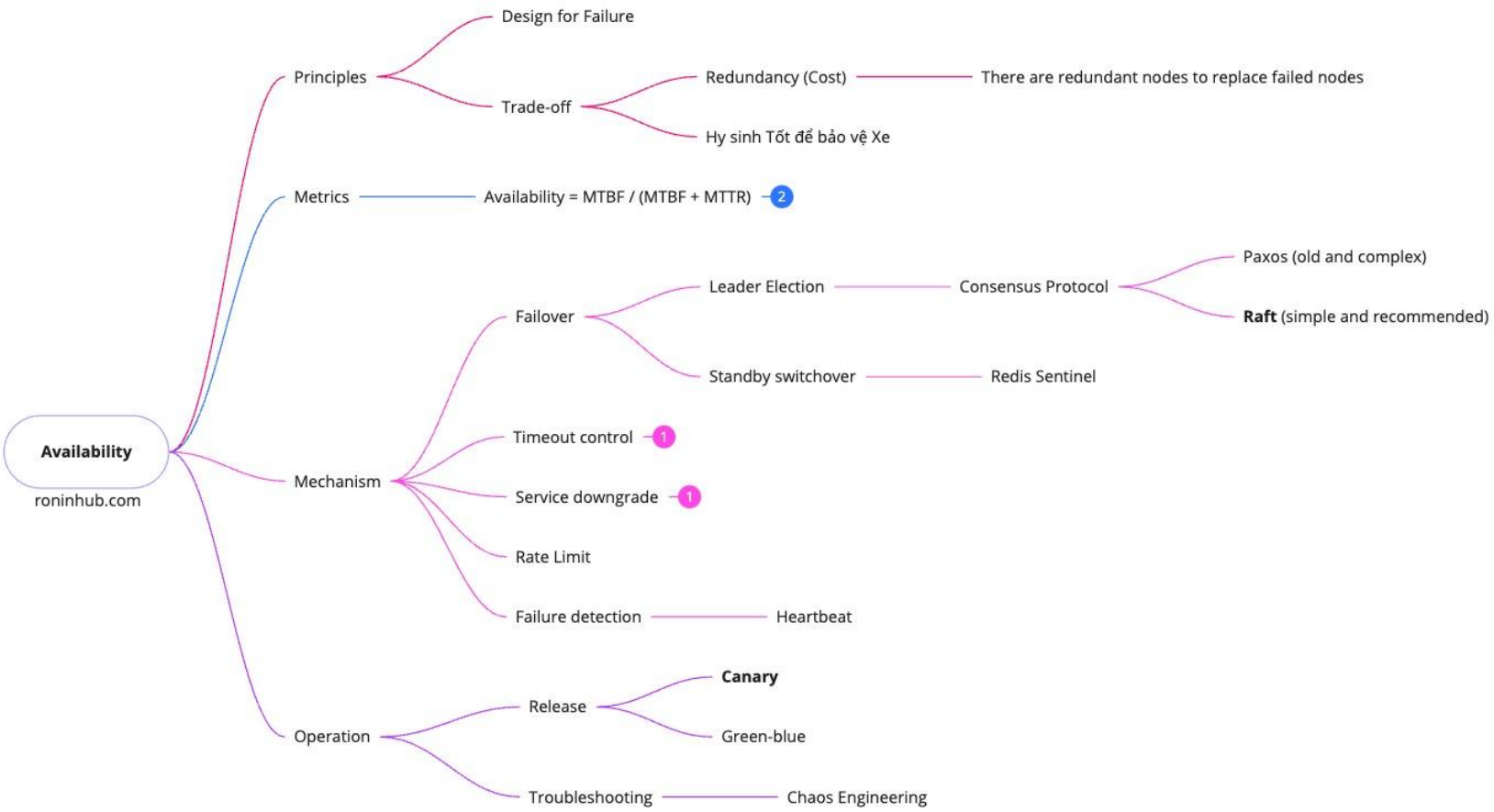
- Bottleneck groups can be:
  - **CPU**
    - *Context switches*
    - *IO waits*
  - **Memory**
    - *Size*
  - **Disk**
    - I/O:
      - IOPS depends on hardware
      - *Sequential I/O >> Random I/O*
    - Fragmentation
  - **Network**
    - Bandwidth
    - *I/O models*

Concurrency Principle

- For one core, executing A and B **sequentially will always be faster than** executing A and B **concurrently** through time-slicing.



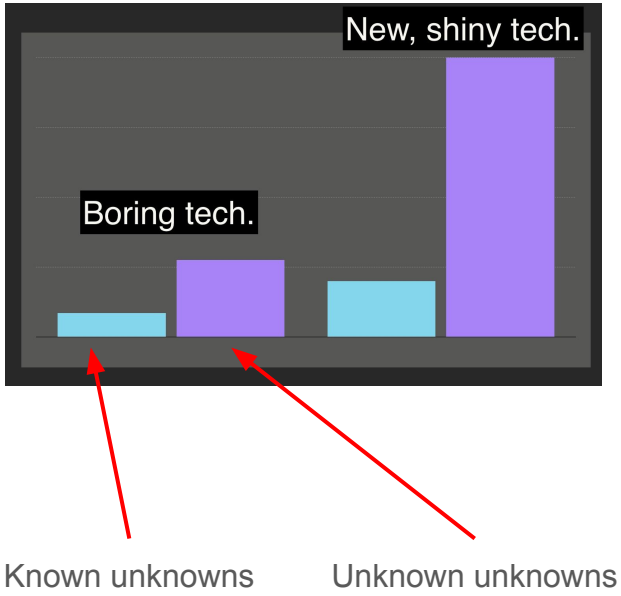
Goal: Availability


Availability
roninhub.com
Principles
Design for Failure
Trade-off
Redundancy (Cost)
There are redundant nodes to replace failed nodes
Hy sinh Tốt để bảo vệ Xe
Metrics
Availability = MTBF / (MTBF + MTTR)
2
Mechanism
Failover
Leader Election
Consensus Protocol
Paxos (old and complex)
Raft (simple and recommended)
Standby switchover
Redis Sentinel
Timeout control
1
Service downgrade
1
Rate Limit
Failure detection
Heartbeat
Operation
Release
Canary
Green-blue
Troubleshooting
Chaos Engineering

How to Choose Technology?

How to Choose Technology?

- Boring Technologies
  - Have been proven
    → increase known unknowns
    → decrease unknown unknowns
  - Familiar with the team
  - Don’t waste innovation tokens
- Community
- Active Development / Roadmap
  - How to check if it’s active
    - https://openhub.net/



Recap

- Quick Estimation: Approximation + Grouping all powers of 10
- Some trade-offs can be mitigated by resolving couplings
- For one core, executing A and B sequentially will always be faster than executing A and B concurrently through time-slicing.

Homework

- Read about LSM Tree, Bitmap, B+Tree
- Design URL shortener



References

-

Thank you 🙏

